

# Backend + Frontend Assessment

**Duration:** 10–12 Hours

**Tech Stack:** Node.js + PostgreSQL + React/Angular

---

## Objective

The purpose of this assessment is to evaluate:

- Backend architecture and coding standards
- Database design and SQL understanding
- Frontend development skills
- API integration capability
- Authentication and authorization handling
- Code quality and scalability thinking
- Problem-solving and decision-making ability
- Ability to explain technical implementation clearly

## Project Overview

Build a **Mini Lead Management System** where managers can create and assign leads to agents.

The system should support:

- Authentication using JWT
- Role-based access
- Lead management
- Lead assignment
- Frontend Screens for CRUD operations

## Tech Stack Requirements

### Backend

Mandatory:

- Node.js
- Express.js
- PostgreSQL

Preferred Architecture:

- Routes
  - Controllers
  - Services
  - Middleware
  - DB Layer
-

## **Frontend**

- React.js / Angular (Preferred)
  - OR Plain HTML
- 

## **Database**

Mandatory:

- PostgreSQL (If you don't have it locally, Supabase OR other similar platforms can be used as FREE db)

## **Functionality Overview**

### **1. Authentication**

Implement JWT-based authentication.

#### **Required APIs**

- Register User
  - Login User
  - Refresh Token (Optional)
  - Logout
- 

#### **Roles**

Support the following roles:

- Admin
  - Manager
  - Agent
- 

#### **Expectations**

- Password hashing
  - JWT middleware
  - Protected APIs
  - Role-based authorization
- 

### **2. Leads Management Module**

Each lead should contain:

| Field       | Type     |
|-------------|----------|
| name        | string   |
| email       | string   |
| phone       | string   |
| source      | string   |
| status      | string   |
| assigned_to | user     |
| notes       | text     |
| created_at  | datetime |

---

## APIs

- Create Lead
  - Update Lead
  - Delete Lead
  - Get Lead by ID
  - List Leads
- 

## Listing Features

Mandatory:

- Pagination
  - Search
  - Sorting
  - Filtering by status/source
- 

## 3. Leads Assessment Logic

Implement auto-assignment logic.

### Requirement

When a manager creates a lead:

- it should automatically assign to an agent

You may implement:

- Round-robin assignment OR
  - Least-loaded agent assignment
- 

## Expectations

Your solution should consider:

- scalability
  - concurrency
  - future extensibility
- 

#### **4. Activity Logs**

At the end, observability and monitoring matters the most. Track important activities:

- Lead Created
  - Lead Updated
  - Lead Assigned
  - Status Changed
- 

#### **5. Third Party API Integration**

Integrate at least ONE external API as the utilities function.

- Email sending API
  - Random user/company enrichment API
  - Webhook API
  - SMS API
  - Geolocation API
- 

### **Expectations**

- Proper async handling
  - Error handling
  - Environment variable usage
- 

### **Frontend Screens**

- Login Page
  - Dashboard
  - Lead Listing (Tabular View with Pagination)
  - Create/Edit Lead Form
  - Lead Details View
- 

#### **Expectations**

Mandatory:

- Form validations
- API integration
- Reusable components
- Loading states
- Error handling
- Clean UI / Coding

Preferred:

- Responsive design (Bootstrap CSS)
- State management
- Proper folder structure

---

## **Database Expectations**

- Proper normalization
- Foreign keys
- Indexes where appropriate

Provide:

- SQL schema OR
- migration files

---

## **Non-Functional Expectations**

We will evaluate:

- Folder structure
- Naming conventions
- Error handling
- Validation handling
- Scalability thinking
- Reusability
- Clean code practices

---

## **Deliverables**

## **1. Source Code**

Provide:

- GitHub repository link

Repository must include:

- backend
  - frontend
  - README
- 

## **2. Readme.md**

README should contain:

- setup instructions
  - environment configuration
  - database setup
  - how to run frontend/backend
  - API documentation
  - assumptions made
  - tradeoffs considered
- 

## **3. Database Design**

- ER Diagram OR
  - schema explanation
- 

## **4. Architecture Explanation (Mandatory)**

Create a PPT/PDF (5–10 slides preferred) explaining:

- Project architecture
  - Folder structure
  - Authentication flow
  - Database design decisions
  - Lead assignment logic
  - Scalability considerations
  - Challenges faced
  - Improvements possible with more time
- 

## **5. AI Usage Disclosure (Mandatory)**

Provide a small document mentioning:

- Which AI tools were used
- What they were used for
- What modifications were done manually

We are not evaluating whether AI was used.

We are evaluating engineering understanding and implementation decisions.

---

## **Bonus Points (Optional)**

Any of the following will earn bonus points:

- Docker setup
- Unit testing
- Swagger/OpenAPI documentation
- WebSockets/live updates
- Redis caching
- Background jobs/queues
- Rate limiting
- Audit middleware
- Deployment link